Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis

Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Zusammenfassung

Dies ist eine Zusammenfassung der Arbeit.

Abstract

This is the abstract.

Contents

List of Figures

List of Tables

1 Einleitung

This chapter contains an overview of the topic as well as the goals and contributions of your work.

1.1 Motivation

1.2 Ziele der Arbeit

1.3 Aufbau der Arbeit

Here you describe the structure of the thesis. For example:
In Kapitel 2 werden grundlegende Methoden für diese Arbeit vorgestellt.

2 Basics

2.1 Definitions

Definition 1 ***Multi-valued planning task.*** A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ has a finite domain x^d and in every state, x takes exactly one value from its domain.
- s_0, the **initial state**, is a total function, s.t. $s_0(x) \in x^d$ for each $x \in \mathcal{F}$
- s_*, the **goal conditions**, is a partial function, s.t. $s_*(x) \in x^d$, for each $x \in dom(s_*)$
- $\mathcal{O}$ is a finite set of operators, also called **actions**. We define an action as $a = \langle a^c, a^e \rangle$, where a^c is the precondition of a and a^e is the postcondition of a. Both a^c and a^e are partial variable assignments over $\mathcal{F}$.

Example 1. Consider the following example for a plan to leave the house:

- $\mathcal{F} = \{door, location\}$ with $door^d = \{open, closed\}$, $location^d = \{inside, outside\}$
- Initial state: $s_0 = \{door{=}closed, location{=}inside\}$
- Goal: $s_* = \{door{=}closed, location{=}outside\}$
- Actions: $\mathcal{O} = \{openDoor, closeDoor, goOut\}$, where

$$openDoor = \langle \{door{=}closed\}, \{door{=}open\} \rangle$$
$$closeDoor = \langle \{door{=}open\}, \{door{=}closed\} \rangle$$
$$goOut = \langle \{location{=}inside, door{=}open\}, \{location{=}outside\} \rangle$$

Definition 2 ***Successor state.*** Let s be state and $a \in \mathcal{O}$ an action.
The *successor state* $o(a, s)$, obtained by applying action a in state s, is defined if the precondtion of a holds, i.e., $a^c(x) = s(x)$ for all $x \in dom(a^c)$. Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in dom(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

Remark. Let $A_n = \langle a_1, \ldots, a_n \rangle$ be a sequence of actions with length n. We define the application of A_n to a state s recursevily by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$.

Definition 3 ***Sequential plan.*** A *sequential plan* is a sequence of actions $A_n = \langle a_1, \ldots, a_n \rangle$, s.t. $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for all $x \in dom(s_*)$.

Example 2. Consider the planning task from the previous example. A sequential plan would be $A = \langle openDoor, goOut, closeDoor \rangle$. Since

$$\begin{aligned} o(A, s_0) &= o(\langle openDoor, goOut, closeDoor \rangle, \{door{=}closed, location{=}inside\}))) \\ &= o(\langle goOut, closeDoor \rangle, \{door{=}open, location{=}inside\})) \\ &= o(closeDoor, \{door{=}open, location{=}outside\}) \\ &= \{door{=}closed, location{=}outside\} = s_* \end{aligned}$$

To get to parallel actions and plans, we need the definition of confluent actions.

Definition 4 ***confluent.*** Let $A = \{a_1, \ldots, a_n\} \subseteq \mathcal{O}$ be a set of actions.
A is *confluent* $:\Longleftrightarrow a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in dom(a_i^e) \cap dom(a_j^e)$

Remark. A is confluent $\Leftrightarrow B$ is confluent for all $B \subseteq A$

Example 3. In our example $\{openDoor, goOut\}$ and $\{closeDoor, goOut\}$ are confluent sets.
We extend our example with the fluents *light* and *key* with $light^d = key^d = \{0, 1\}$ and two actions *lightsOut* and *getKey*:

$lightsOut = \langle \{location{=}inside, light{=}1\}, \{light{=}0\} \rangle$

$getKey = \langle \{location{=}inside, key{=}0\}, \{key{=}1\} \rangle$

Additionally, we modify the precondition: $openDoor^c = \{door{=}closed, key{=}1\}$. Our new goal is $s_* = \{location{=}outside, door{=}closed, light{=}0\}$.
Now for example $\{getKey, openDoor, goOut, lightsOut\}$ and $\{getKey,\ closeDoor,\ goOut,\ lightsOut\}$ are also confluent sets.

Definition 5 ***Parallel representations.*** Let s be a state and $A = \{a_1, \ldots, a_n\}$ be a confluent set of actions. Then A is

(i) $\forall$*-step serializable* in $s :\Longleftrightarrow o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)} \rangle, s)$ is defined for all permutations π

(ii) $\exists$*-step serializable* in $s :\Longleftrightarrow o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)} \rangle, s)$ is defined for some permutation $\pi[t]$ and $a^c(x) = s(x)$ for each $a \in A$ and $x \in dom(a^c)$

(iii) *relaxed* $\exists$*-step serializable* in $s :\Longleftrightarrow o(\langle a_{\pi(1)}, \ldots, a_{\pi(n)} \rangle, s)$ is defined for some[t] permutation π

Remark. A is $\forall$-step serializable $\Rightarrow A$ is $\exists$-step serializable $\Rightarrow A$ is relaxed $\exists$-step serializable.

Definition 6 ***Parallel Plans.*** Let $A = \langle A_1, \ldots, A_n \rangle$ be a sequence of confluent sets of actions. A is a $\forall$*-step,* $\exists$*-step or relaxed* $\exists$*-step plan* iff

- $s_n(x) = s_*(x)$ for each $x \in dom(s_*)$
- A_i is $\forall$-step, $\exists$-step or relaxed $\exists$-step serializable in s_{i-1} for $1 \leq i \leq n$, where
  $s_i(x) = \begin{cases} a^e(x) & \text{if } x \in dom(a^e) \\ s_{i-1}(x) & \text{else} \end{cases}$ for each $a \in A_i$

Example 4. In our modified example we have:

- $\forall$-step plan: $\langle\{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\}\rangle$
- $\exists$-step plan: $\langle\{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\}\rangle$
- relaxed $\exists$-step plan: $\langle\{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\}\rangle$

2.1.1 ASP

Our little example looks like this in ASP:

```
fluent(door).         fluent(location).          fluent(light).   fluent(key).
value(door, closed). value(location, inside).  value(light, 0). value(key, 0).
value(door, open).    value(location, outside). value(light, 1). value(key,1).

init(door, closed). init(location, inside).  init(light, 1). init(key, 0).
goal(door, closed). goal(location, outside). goal(light, 0).

action(openDoor).              action(closeDoor).              action(goOut).
prec(openDoor, door, closed). prec(closeDoor, door, open).    prec(goOut, location, inside).
prec(openDoor, key, 1).       post(closeDoor, door, closed). prec(goOut, door, open).
post(openDoor, door, open).                                  post(goOut, location, outside).

action(lightsOut).                 action(getKey).
prec(lightsOut, location, inside). prec(getKey, location, inside).
prec(lightsOut, light, 1).         prec(getKey, key, 0).
post(lightsOut, light, 0).         post(getKey, key, 1).
```

Combining these facts with encodings, gives us the corresponding *sequential, $\forall$-step, $\exists$-step* or *relaxed $\exists$-step plan.*

```
#include <incmode>.
holds(X, V, 0) :- init(X, V).
#program check(t).
:- query(t), goal(X, V), not holds(X, V, t).
#program step(t).
{holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
{ occurs(A, t) } :- action(A) .
:- occurs(A, t), post(A, X, V), not holds(X, V, t).
:- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).

```

```
11 #show.
12 #show occurs(A, t) : occurs(A, t).
```

```
1 #program step(t).
2 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
3 :- #count{A : occurs(A, t)} > 1.
```

Bibliography

Helmert, M. (2006). "The Fast Downward Planning System". In: *Journal of Artifical Intelligence Research* 26, pp. 191–246.